A Sheath Boundary Condition for JOREK

From the plasma physics to the finite elements, with the numerics explained from first principles

Máté Szűcs

26 August 2026

Abstract

The electric potential at a divertor target is set by the Debye sheath, and the resulting $\boldsymbol{E} \times \boldsymbol{B}$ drift is a candidate mechanism for the in–out divertor density asymmetry. JOREK's reduced-MHD `model600` previously froze the potential at the wall, which forbids that physics. This paper documents a boundary condition that instead lets the plasma determine the wall potential through the Langmuir current–voltage characteristic.

Most of the difficulty was not in the physics but in the numerics, and that is where this document spends its time. Imposing the characteristic *pointwise at boundary nodes* satisfies it there to one part in 5000 and still leaves a 33–48 % error in the integrated wall current. Understanding why — and what to do instead — requires knowing what a finite-element solution actually is, what a boundary condition means inside such a code, and what it means for an equation to be "satisfied" when the unknowns are coefficients rather than values. Sections 3 to 5 build those ideas from scratch, assuming no finite-element background.

The resulting condition closes the wall current budget to four significant figures on both targets, needs no tuning parameter, and has run 3900 time steps to wall-clock timeout on an ASDEX Upgrade X-point geometry.

Contents

1 How to read this

This is written for someone who knows plasma physics and tokamak edge transport, and who has used a finite-element code without necessarily knowing what one is doing internally. That is the gap this document tries to close.

- **Sections 1–2** are the physics. If you know sheath physics you can skim, but Section 2.5 (units and signs) repays care: an error there inverts every later conclusion and is invisible in most diagnostics.
- **Section 3 is a crash course in finite elements**, built around the objects this boundary condition manipulates. Nothing in it is sheath-specific. If you have never had to think about what a degree of freedom *is*, start here.
- **Sections 4–5** are the heart: a natural implementation that fails, why, and the one that works.
- **Sections 6–7** are the code and the results.
- **Section 8** is a five-minute verbal version for explaining this to someone else.
- A **glossary** of numerics vocabulary is at the end.

Blue boxes introduce a numerical concept. Yellow boxes restate something in plain language. Red boxes flag a trap that cost real time.

2 The physics: why the wall potential must be free

2.1 The sheath in one page

A material surface in contact with plasma charges up. Electrons are far lighter than ions and therefore far faster — their thermal speed exceeds the ion sound speed by roughly $\sqrt{m_i/m_e} \approx 60$ in deuterium — so a neutral surface initially collects much more electron than ion current. It charges negatively until the resulting potential barrier reflects enough electrons to balance the ion flux.

The layer across which this drop occurs is the *Debye sheath*. It is a few Debye lengths thick, which in divertor conditions means tens of microns — smaller than any cell in an MHD mesh by four or five orders of magnitude. It cannot be resolved, so it must be represented as a *boundary condition* relating the current through the surface to the potential at the surface.

2.2 Why this matters beyond the current budget

A floating surface settles at

$$\Phi_f = \Lambda \frac{kT_e}{e}, \tag{1}$$

proportional to the local electron temperature, with $\Lambda \approx 3$. A divertor target has a strongly varying T_e across the strike point — tens of eV in the far scrape-off layer, a few eV or less on a detaching leg — so it also has a strongly varying *potential*. A radially varying potential is a radial electric field $E_r = -\partial\Phi/\partial r$ at the target, and an electric field perpendicular to $\boldsymbol{B}$ is an $\boldsymbol{E} \times \boldsymbol{B}$ drift.

In a diverted geometry that drift runs through the private flux region, transporting particles between the inner and outer targets. It is one of the standard candidate explanations for the high-field-side high-density (HFSHD) asymmetry, and enabling it is why this work exists.

> The sheath sets the wall potential; the wall potential follows T_e; a varying potential is an electric field; an electric field across $\boldsymbol{B}$ pushes plasma sideways. That sideways push is the physics we want.

2.3 What was there before, and why it forbids the physics

The previous condition was $u = 0$ on the wall, where u is JOREK's velocity stream function and $\Phi = -F_0 u$ (Section 2.5). That states $\Phi = 0$ everywhere on the wall.

This is not a harmless gauge choice. Once a sheath is present the absolute potential means something: $\Phi = 0$ is $X = -\Lambda$ in the characteristic below, i.e. deep *electron saturation*, the wall drawing near-maximal electron current everywhere. And a constant Φ has $\partial\Phi/\partial r = 0$ by construction, so E_r vanishes identically and the drift mechanism is structurally inaccessible. No amount of resolution or run time recovers it.

3 The sheath current–voltage law

3.1 The characteristic

For a Maxwellian plasma at a wall the ion current is saturated at the Bohm value while the electron current is retarded by a Boltzmann factor. The net current density along the field, referenced to the wall, is the Langmuir probe characteristic (Stangeby [1], eq. 2.68):

$$j_\parallel = j_{\mathrm{sat}} \left(1 - e^{-X}\right), \qquad X \equiv \frac{e\Phi}{kT_e} - \Lambda, \qquad \Phi = V_{\mathrm{sheath}} - V_{\mathrm{wall}}. \tag{2}$$

Three points are worth memorising, and Figure 1 shows them.

- $X = 0$, i.e. $\Phi = \Lambda kT_e/e$: **the floating potential**, zero net current. Where an electrically isolated surface sits.
- $X \to +\infty$: **ion saturation**. The current tends to j_{sat} and no further — raising the potential cannot push more ion current through.
- $X = -\Lambda$, i.e. $\Phi = 0$ (grounded wall): **electron saturation**, drawing $j_{\mathrm{sat}}(1 - e^{\Lambda}) \approx -19\, j_{\mathrm{sat}}$. Note the asymmetry: the electron branch is nineteen times larger and grows exponentially.

3.2 The floating coefficient Λ

Λ is the log of the ratio of the two thermal fluxes, $\Lambda_0 = \ln\sqrt{m_i/2\pi m_e} \simeq 3.19$ for deuterium, with an optional correction for the ion contribution to the sound speed, $\Lambda = \Lambda_0 - \frac{1}{2}\ln[\gamma(T_i + T_e)/T_e]$. The correction is not cosmetic: at $T_i = T_e$ it gives 2.4 rather than 3.0, which is 12 V at $T_e = 20$ eV.

The saturation current follows the Bohm condition, $j_{\mathrm{sat}} = e\, n\, c_s\, g(b_n)$ with $c_s = \sqrt{\gamma(T_i + T_e)/m_i}$, where $g(b_n)$ is the Chodura–Riemann obliqueness factor carrying the sign of $\boldsymbol{B}\cdot\boldsymbol{n}$. It is the *same* factor the Mach-1 boundary condition uses for the parallel outflow, which is what makes current and particle flux mutually consistent at the same surface — and why `bcs%mach1` must be on wherever this condition is used.

3.3 Forward or inverse? A choice that decides everything

Equation (2) reads in either direction, and the two readings behave completely differently.

The *inverse* form solves for potential given current, $e\Phi/kT_e = \Lambda - \ln(1 - j_\parallel/j_{\mathrm{sat}})$. This is the intuitive statement — "the wall floats to whatever potential passes the delivered current" — and it is how the first implementation was written. It is also singular exactly where divertor plasmas

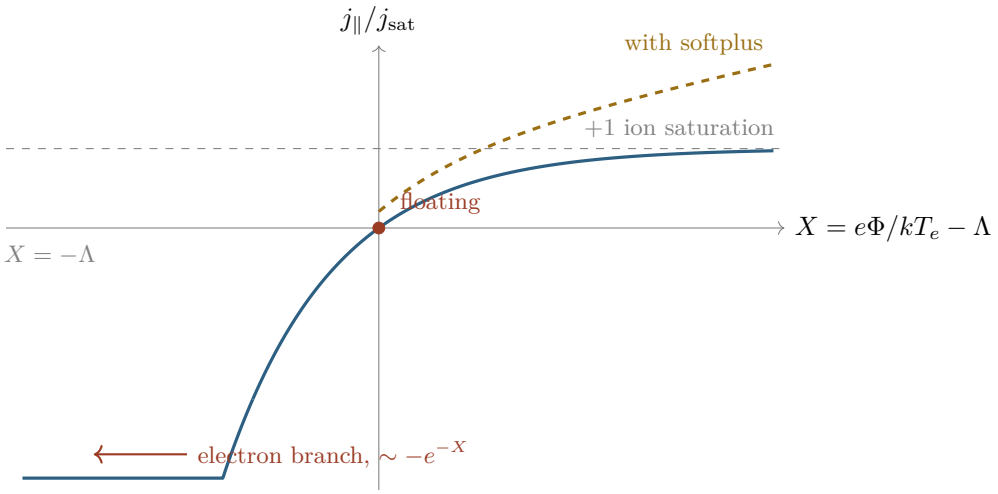

Figure 1: The characteristic. Solid: eq. (2). Dashed: with the softplus of Section 2.4, which gives the ion branch a finite slope so every demanded current is reachable. The electron branch is unbounded and exponential; that asymmetry drives several later numerical choices.

live: as $j_\parallel \to j_{\mathrm{sat}}$ the logarithm diverges, and for $j_\parallel > j_{\mathrm{sat}}$ there is *no solution at all*. A restart built without this boundary condition has no reason to respect $|j_\parallel| \le j_{\mathrm{sat}}$, and in practice does not.

The *forward* form used here, $j_\parallel = j_\parallel(\Phi)$, is single-valued for every Φ, and its derivative $\partial j_\parallel/\partial\Phi \propto e^{-X}$ tends smoothly to zero on the ion side. Nothing needs clipping and the linearisation never becomes singular.

> Same equation, opposite numerical character. "What current flows at this potential?" always has an answer. "What potential passes this current?" has none once the plasma demands more than the sheath can carry — which is the normal state of a divertor.

3.4 Two regularisations, and why each is needed

Finite conductance at ion saturation. The forward characteristic saturates *exactly* at j_{sat}, so a plasma delivering even a few percent more has no consistent boundary state and the solver drives Φ upward forever chasing it. The fix gives the ion branch a finite differential conductance:

$$f(X) = \left(1 - e^{-X}\right) + s\,\ln\!\left(1 + e^{X}\right), \qquad z_j^{\mathrm{sh}} = z_j^{\mathrm{sat}}\, f(X), \tag{3}$$

with $s =$ `sheath_sat_slope`. The added *softplus* term is exponentially small for $X \ll 0$, so the electron branch and the floating potential are untouched, while f becomes unbounded above and every demanded current is reachable at finite X. It is C^∞.

A limiter on the electron branch. Going the other way, f grows like $-e^{-X}$ without bound, so a smooth floor is applied to the exponent,

$$X_{\mathrm{lim}} = X_{\mathrm{min}} + \delta X\,\ln\!\left(1 + e^{(X - X_{\mathrm{min}})/\delta X}\right), \tag{4}$$

C^1, monotone, tending to the identity for $X \gg X_{\mathrm{min}}$. The default $X_{\mathrm{min}} = -3 \approx -\Lambda$ corresponds to $\Phi \ge 0$.

Trap: $X_{\min}$ is a clamp, not a restoring force

It is tempting to lower $X_{\min}$ to "give the solver more room". That is backwards. $X_{\min}$ is what *stops* the electron current growing: at $X_{\min} = -3$ the floor on f is -19, at $X_{\min} = -6$ it is -445. Measured, lowering it made a failing case fail faster.
A second problem lurks below the clamp. Once $X \ll X_{\min}$, eq. (4) gives $\mathrm{d}X_{\lim}/\mathrm{d}X \to 0$, hence $\partial z_j^{\mathrm{sh}}/\partial u \to 0$: the sheath stops responding to the potential at all. If nothing else constrains u there, it is in free fall.

Two numerical details, both easy to get wrong. $1 - e^{-X}$ cancels catastrophically near $X = 0$ — the floating condition, i.e. the regime of interest rather than a corner case — and Fortran has no `expm1`, so a four-term Taylor series is used below $|X| < 10^{-4}$. And all exponentials are evaluated only inside $|X| \le 30$, with asymptotic branches outside, so a debug build with `-ffpe-trap=overflow` survives any state the solver passes through.

3.5 JOREK units and the sign convention

Short, and worth reading twice.

JOREK's reference paper (Hölzl *et al.* [2], eq. 26) defines $u = \Phi/F_0$ with the ansatz $\boldsymbol{v}_{\mathrm{pol}} = -R\nabla u \times \boldsymbol{e}_\phi$. The `model600` element matrix implements $\boldsymbol{v}_{\mathrm{pol}} = +R\nabla u \times \boldsymbol{e}_\phi$. The code's u is therefore *minus* the paper's:

$$\boxed{\Phi = -F_0\, u} \tag{5}$$

The same inversion applies to ψ and to the current: the code's $z_j = \Delta^*\psi$ is minus the paper's j. With n_0 the central density, $\rho_0 = n_0 m_i$:

$$\frac{e\Phi}{kT_e} = \frac{\frac{1}{2}a_n u - v_w}{T_e}, \qquad a_n = -\frac{2eF_0\sqrt{\mu_0\rho_0}}{m_i}, \qquad v_w = e\,V_{\mathrm{wall}}\,\mu_0\,n_0, \tag{6}$$

and the saturation current in code variables (Artola *et al.* [3]) is

$$z_j^{\mathrm{sat}} = c_{\mathrm{sat}}\,\rho\,g(b_n)\,\frac{c_s}{|\boldsymbol{B}|}, \qquad c_{\mathrm{sat}} = -\tfrac{1}{2}a_n, \qquad c_s = \sqrt{\gamma(T_i + T_e)}. \tag{7}$$

This is derived once, in one module, and used by all three consumers — assembly, diagnostic, and the legacy nodal route — so they cannot drift apart.

Trap: a shared normalisation cancels in its own check

The diagnostic reports I_{sheath} and $I_{\mathrm{Ampère}}$ and their agreement is the main acceptance test. But both are computed as $\oint z_j\,(\boldsymbol{B}\cdot\boldsymbol{n})/(F_0\mu_0)\,\mathrm{d}S$, so an error in *that* conversion cancels in the ratio. A factor of two in a_n would pass every check the code currently prints. Real verification needs a test that leaves the code's normalisation entirely — Section 7.4.

3.6 Temperature floors

The characteristic divides by T_e, so it is sensitive to the positivity floor on temperature. JOREK's global floor `T_min_neg` exists to keep every *other* term well behaved and is normally set well above divertor temperatures: at 3×10^{-5} the knee sits at 1.47 eV. A real target profile running $0.40 \to 0.47$ eV is then delivered to the sheath as $0.91 \to 0.93$ eV — a 19 % contrast compressed to 2 %.

Since $\Phi_f = \Lambda k T_e/e$ inherits that compression, the target's radial potential gradient — precisely the E_r this work exists to produce — is flattened before the boundary condition ever sees it. `T_min_sheath` therefore provides a separate floor with the same functional form, so the sheath can see a colder plasma than the rest of the model. This is a physics knob, not a numerical convenience.

4 A crash course in finite elements

Everything from here on depends on knowing what a finite-element code is actually storing and solving. This section builds that up from nothing. It is not sheath-specific and nothing in it is deep — but the boundary condition cannot be explained, let alone debugged, without it.

4.1 The solution is a function, not a table of values

The first and most important idea. When a finite-difference code says "the density at grid point k", it means a number, and the solution between grid points is undefined — you would interpolate if you needed it. A finite-element code is different: it represents the solution as a *continuous function defined everywhere*, written as a sum

$$\rho(\boldsymbol{x}) = \sum_a \rho_a \, N_a(\boldsymbol{x}). \tag{8}$$

The $N_a(\boldsymbol{x})$ are fixed, known *basis functions*, chosen once when the mesh is built. The ρ_a are numbers — the *degrees of freedom*, or DOFs — and they are what the code actually solves for.

> **Basis functions**
>
> Think of a Fourier series, $f(x) = \sum_k c_k e^{ikx}$: you solve for the coefficients c_k, not for f at particular points, and f is then defined everywhere. Finite elements are the same idea with *local* basis functions instead of global waves — each N_a is a bump that is nonzero only over the handful of elements touching node a, and zero everywhere else.
> Locality is the whole point: it makes the matrices sparse. Global smooth functions would give a dense matrix and be unusable at this size.

So the unknown vector is a list of coefficients. This has an immediate consequence that will matter enormously in Section 4:

> Controlling a degree of freedom is not the same as controlling the solution. The DOFs are coefficients in a sum. Fixing some of them pins the function at particular places, in particular ways — and leaves it free everywhere else.

4.2 Degrees of freedom: JOREK has four per node, per variable

For the simplest elements, the coefficient ρ_a happens to equal the value of ρ at node a, and the distinction above feels pedantic. JOREK does not use those elements.

JOREK uses *bicubic Bézier* elements, which are C^1-continuous: not only the function but its first derivatives match across element boundaries. Achieving that requires more information per node than a single value. Each node carries **four** degrees of freedom per variable:

$$\underbrace{\rho}_{\text{value}}, \qquad \underbrace{\partial_s\rho, \ \partial_t\rho}_{\text{two first derivatives}}, \qquad \underbrace{\partial_s\partial_t\rho}_{\text{mixed second}} \tag{9}$$

where (s, t) are the element's local coordinates. Four DOFs $\times$ four nodes $= 16$ coefficients describe the variable inside one element, which is exactly what a bicubic polynomial needs.

> **Why C^1 matters here**
>
> Reduced MHD contains operators like $\Delta^*\psi$ and Poisson brackets $[u, \psi]$ — second derivatives and products of first derivatives. With only C^0 continuity, first derivatives jump across element faces and second derivatives are meaningless there. C^1 elements make the physics

representable at the price of four DOFs per node instead of one.

"The value of u at node 17" is *one of four numbers* the code stores at node 17. The other three say how u is sloping there. All four affect the solution between the nodes.

4.3 The trace: what a boundary edge can see

Now the key geometric fact this entire implementation rests on.

Take one element with a face on the wall. Restrict the solution to that edge — the resulting one-dimensional function is called the *trace*. Which of the element's sixteen coefficients affect it?

Only four. Along an edge of constant t (say), the trace is a cubic in s, and a cubic is fixed by four numbers: the value and the tangential derivative ∂_s at each of the two endpoints. The *normal*-derivative DOF ∂_t and the mixed DOF $\partial_s\partial_t$ have basis functions that vanish identically on that edge. They control how the solution leaves the wall, not what it does along it.

The trace space

The trace of the solution on a boundary edge is a cubic determined by four DOFs: value and tangential derivative at each endpoint. The set of all functions reachable this way is the *trace space*. It is the complete answer to "what shapes can the solution take along this wall?"

This is what makes the boundary condition tractable. Any statement about the solution on the wall only ever needs to reference those four DOFs per edge — not all sixteen.

4.4 Strong form, weak form, and where surface terms come from

A PDE like $\nabla^2\phi = f$ is in *strong form*: it asserts an equality at every point. But the finite-element solution is a finite sum, eq. (8), and cannot satisfy an equation at every point in general. So we ask for less.

Multiply by a *test function* N_a and integrate over the domain:

$$\int_\Omega N_a\,\nabla^2\phi\,\mathrm{d}V = \int_\Omega N_a\,f\,\mathrm{d}V \qquad \text{for every } a. \tag{10}$$

This gives one equation per DOF — exactly as many equations as unknowns. Now integrate the left side by parts:

$$\int_\Omega N_a\,\nabla^2\phi\,\mathrm{d}V = -\int_\Omega \nabla N_a\cdot\nabla\phi\,\mathrm{d}V \;+\; \underbrace{\oint_{\partial\Omega} N_a\,\frac{\partial\phi}{\partial n}\,\mathrm{d}S}_{\text{surface term}}. \tag{11}$$

Two things have happened. The second derivative has become a first derivative, which lowers the smoothness the basis must have. And a *surface term* has appeared, integrated over the boundary.

Where boundary conditions live

That surface term is the natural home of a boundary condition. If you know $\partial\phi/\partial n$ on the wall, substitute it and you are done — that is a *natural* (Neumann) condition, and it requires no special machinery.

If instead you want to fix ϕ itself, the surface term does not help: you must overwrite equations. That is an *essential* (Dirichlet) condition. The distinction matters because a condition of the wrong type for the equation you attach it to either does nothing or does

damage.

Two consequences that will return:

- **An equation assembled in strong form has no surface term to replace.** Adding one injects a spurious source rather than supplying a missing piece. The `model600` vorticity equation is in this category, which is why the sheath is not attached there.
- **If a surface term is required but refused, the boundary equation is incomplete.** The $z_j = \Delta^*\psi$ equation is integrated by parts and its surface term is refused by the assembly for reasons in Section 5.2. That is harmless while a Dirichlet overwrites those rows anyway — and becomes a real problem the moment the Dirichlet is released.

4.5 What a boundary condition *is*, inside the code

Assembly produces a linear system $\mathsf{A}\,\boldsymbol{x} = \boldsymbol{b}$, one row per DOF. Row a is the weak-form equation obtained with test function N_a.

Imposing a Dirichlet condition means *replacing* row a with the statement you want, typically $x_a =$ value. JOREK does this with a penalty rather than by clearing the row: it overwrites one diagonal entry with `zbig` $= 10^{12}$, so that entry dominates everything else in the row by about eighteen orders of magnitude and the row effectively reads $x_a \approx$ value.

A "boundary condition" in a finite-element code is a modification to specific rows of a matrix. Which rows, and what you put in them, *is* the boundary condition. There is nothing else to it.

This framing makes the design question sharp. To impose the sheath we must decide: *which rows*, and *what goes in them*. Sections 4 and 5 are entirely about getting those two answers right.

4.6 Nonlinear conditions: Newton, Jacobians and columns

The sheath law is nonlinear — z_j^{sh} depends on u through exp. A linear solver cannot take a nonlinear row, so it must be linearised.

Given a current state $\boldsymbol{x}$, write the condition as a residual $F(\boldsymbol{x}) = 0$ and expand:

$$F(\boldsymbol{x} + \mathrm{d}\boldsymbol{x}) \approx F(\boldsymbol{x}) + \sum_k \frac{\partial F}{\partial x_k}\,\mathrm{d}x_k. \tag{12}$$

Setting this to zero gives the Newton step: solve $\mathsf{J}\,\mathrm{d}\boldsymbol{x} = -F$, where the *Jacobian* J holds the partial derivatives. Its entries occupy *columns* of the row — one column per variable the residual depends on.

For the sheath, z_j^{sh} depends on u, ρ, T_i, T_e (and trivially on z_j itself), so the row has columns for those five and no others. Two omissions are worth stating explicitly: $v_\parallel$ and w appear *not at all*, because `sheath_current` takes neither, so their derivatives are exactly zero. And ψ is omitted *by choice*: $z_j^{\mathrm{sat}} \propto 1/|\boldsymbol{B}|$ and $g(b_n)$ do depend on the field, but no $\partial z_j^{\mathrm{sh}}/\partial\psi$ is returned, so the boundary geometry is frozen within a Newton iteration. That makes the scheme *quasi-Newton* — it converges to the right answer, just not at the full quadratic rate.

Trap: damping the Jacobian magnifies the step

When a Newton step overshoots, the instinct is to shrink the Jacobian entries. That is exactly backwards. The step solves $\mathsf{J}\,\mathrm{d}\boldsymbol{x} = -F$, so multiplying J by $\alpha < 1$ multiplies $\mathrm{d}\boldsymbol{x}$ by $1/\alpha$ — a *bigger* step. To limit a step you bound F, the right-hand side. This is the

reasoning behind the trust region in Section 5.4.

4.7 Conditioning: why the numbers span ten orders of magnitude

A last piece of vocabulary. Consider the boundary mass matrix, whose entries are $\int N_a N_b \, \mathrm{d}S$ over the wall. How big are they?

It depends which DOFs a and b are. A value-type basis function is $O(1)$ on an edge of length h, so a value–value entry scales as h. A derivative-type basis function carries a factor of the element size, so a value–derivative entry scales as h^2 and a derivative–derivative entry as h^3. At $h \sim 1$ mm those three differ by six orders of magnitude *within the same matrix block*, before any physics is involved.

Conditioning

A matrix is *ill-conditioned* when its entries (more precisely its eigenvalues) span a huge range. Solvers lose precision in proportion, and — more relevant here — any scheme that applies a single uniform coefficient across such a block cannot work: whatever value makes one row behave makes another either negligible or overwhelming.
The measured spread on this mesh is $D_{\max}/D_{\min} = 1.62 \times 10^{-3}/6.34 \times 10^{-14} = 2.6 \times 10^{10}$. Ten orders of magnitude, from geometry alone.

"Derivative" degrees of freedom are numerically tiny compared with "value" ones, because they carry factors of the mesh size. Any recipe of the form "multiply the whole boundary block by β" is doomed before you start choosing β.

5 The failure that motivated everything

5.1 The obvious implementation

The natural way to impose $z_j = z_j^{\mathrm{sh}}$ is to write it at each boundary *node*: find the DOF that holds the value of z_j at that node, replace its row with the linearised condition

$$\mathrm{d}z_j - \sigma \sum_k \frac{\partial z_j^{\mathrm{sh}}}{\partial x_k} \, \mathrm{d}x_k = \sigma \left(z_j^{\mathrm{sh}} - z_j \right), \tag{13}$$

and let the solver do the rest. (σ is an under-relaxation factor; the diagonal is left unscaled so σ is the fraction of the mismatch closed per step.)

This is a perfectly reasonable thing to write, and it is what the nodal route (`bcs%sheath_zj`) does.

5.2 It works — and it doesn't

Measured on its own terms it works beautifully. The pointwise residual

$$\varepsilon_{\mathrm{node}} = \sqrt{\frac{\sum_{\mathrm{nodes}} (z_j - z_j^{\mathrm{sh}})^2}{\sum_{\mathrm{nodes}} (z_j^{\mathrm{sat}})^2}} \tag{14}$$

falls to 1.8×10^{-4}. The condition is satisfied at the nodes to one part in five thousand.

And the physical acceptance test fails badly. That test is that the current the sheath passes must equal the current the plasma delivers:

$$I_{\mathrm{sheath}} = \oint_{\Gamma_{\mathrm{sh}}} \frac{z_j^{\mathrm{sh}} \, (\boldsymbol{B}\cdot\boldsymbol{n})}{F_0 \mu_0} \, \mathrm{d}S \;\stackrel{!}{=}\; I_{\mathrm{Amp}} = \oint_{\Gamma_{\mathrm{sh}}} \frac{z_j \, (\boldsymbol{B}\cdot\boldsymbol{n})}{F_0 \mu_0} \, \mathrm{d}S. \tag{15}$$

It missed by 33–48 % on the inner target. Run after run, under every variation of the characteristic, the gates, and the relaxation.

5.3 The diagnosis

The two statements are simply different, and Figure 2 shows how.

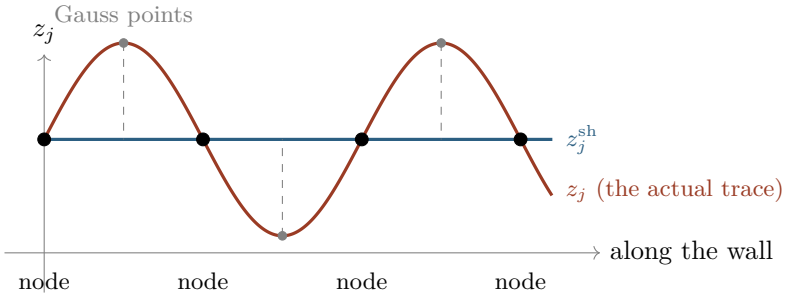

Figure 2: Why the nodal route deceives. The cubic trace passes exactly through the correct value at every node — $\varepsilon_{\mathrm{node}} = 1.8 \times 10^{-4}$ — while being wrong everywhere in between. The currents in eq. (15) are integrals, evaluated at the Gauss points *between* the nodes, so they see the grey dots and not the black ones.

The constraint is imposed at nodes. The currents are integrated at quadrature points between them, where the cubic is unconstrained. A cubic passing through the right endpoint values can integrate to something quite different — and here it did. The Gauss-point residual (same definition, weighted by $\mathrm{d}S$) was measured at ≈ 2: four orders of magnitude larger than the nodal one.

This was confirmed before changing any code, by instrumenting the *weak* residual (Section 5.1) as a pure diagnostic. As $\varepsilon_{\mathrm{node}}$ fell $1.11 \to 0.08 \to 1.8 \times 10^{-4}$, that diagnostic stayed at $\mathcal{O}(1)$. The condition was being satisfied where it was imposed and nowhere else.

> Four numbers per node describe the solution, and the nodal route was controlling one of them. The other three — and hence the shape of the curve between the nodes — were free to do whatever the rest of the physics wanted. Since the currents are integrals over that curve, they were free too.

5.4 The lesson

> **Impose the equation where it is measured**
>
> If an acceptance test is an integral over the boundary, the constraint must be enforced as an integral over the boundary. Enforcing it pointwise and hoping the integral follows is not a weaker version of the same thing — it is a different condition, and it can be satisfied to four decimal places while the quantity you care about is wrong by half.

6 The weak formulation

6.1 The Galerkin residual

We want a statement of "$z_j = z_j^{\mathrm{sh}}$ on the wall" that is an integral. Let $\{N_a\}$ span the boundary trace space (Section 3.3) and Γ_{sh} be the sheath boundary. Define

$$F_a = \int_{\Gamma_{\mathrm{sh}}} N_a \left(z_j^{\mathrm{sh}} - z_j \right) \mathrm{d}S, \qquad D_a = \int_{\Gamma_{\mathrm{sh}}} N_a N_a \, \mathrm{d}S. \tag{16}$$

The condition imposed is $F_a = 0$ for every trace DOF a.

> **What a Galerkin condition means**
>
> $F_a = 0$ for all a says: the mismatch $z_j^{\mathrm{sh}} - z_j$ is *orthogonal* to every function the trace space can represent. It does not say the mismatch is zero — that would be impossible, since z_j^{sh} is not generally a cubic. It says the mismatch has no component the discretisation could have removed.
> This is the natural notion of "satisfied" for a finite-element solution, and it is exactly the right one here: because the test functions N_a are the same ones the currents are integrated against, driving F_a to zero makes the closure (15) hold *by construction* rather than by convergence.

Linearising about the current state, the replacement row for trace DOF a is

$$\underbrace{\int_{\Gamma_{\mathrm{sh}}} N_a N_b \,\mathrm{d}S}_{z_j \text{ column}} \mathrm{d}z_{j,b} \;-\; \sum_{x\in\{u,\rho,T_i,T_e\}} \underbrace{\int_{\Gamma_{\mathrm{sh}}} N_a \frac{\partial z_j^{\mathrm{sh}}}{\partial x} N_b \,\mathrm{d}S}_{\text{state columns}} \mathrm{d}x_b \;=\; F_a. \tag{17}$$

Note there is no θ and no time step: this is an algebraic constraint on the state, not a discretised evolution equation.

Getting this to work required four further ingredients. Each was individually necessary; each is a lesson in the corresponding piece of numerics.

6.2 Ingredient 1: replacement, not penalisation

The obvious way to impose (17) is as a penalty — add βF_a to the existing z_j equation and turn β up until it dominates. This cannot work here, and understanding why is the crux of the whole design.

Recall from Section 3.4 that $z_j = \Delta^*\psi$ is integrated by parts and therefore has a surface term. That surface term is *refused* by the assembly, because its Jacobian needs columns at the normal-derivative DOFs and the trial-function loop emits columns only at the tangential ones.

While `dirichlet%zj` is on, this is harmless: those rows get overwritten anyway. But the weak route *must* release that Dirichlet — otherwise z_j at the wall is frozen and the j–V loop is open (Section 5.7). And the moment it is released, the boundary z_j equation is **incomplete**.

> A penalty adds your condition to an equation that is already there. If that equation is wrong, adding to it does not make it right — you get some blend of the two. Measured: $\beta = 10^{-2}$ left the wrong equation dominant, and $\beta = 1$ produced a period-2 divergence.

Writing the row with `zbig` *annihilates* the incomplete equation instead of adding to it, exactly as every Dirichlet in the code already does.

6.3 Ingredient 2: row normalisation

From Section 3.7, the trace block spans 2.6×10^{10} internally. A single coefficient across it cannot work at any magnitude; $\beta = 10^9$ died in one step.

The fix is to divide each row by its own diagonal:

$$\frac{1}{D_a}\left[\int N_a N_b \mathrm{d}S \,\mathrm{d}z_{j,b} - \sum_x \int N_a \frac{\partial z_j^{\mathrm{sh}}}{\partial x} N_b \mathrm{d}S \,\mathrm{d}x_b\right] = \frac{F_a}{D_a}. \tag{18}$$

Now every row is $O(1)$ regardless of whether it belongs to a value or a derivative DOF, and a single `zbig` makes them all uniformly dominant.

Why this removes the tuning parameter

Before normalisation, "how strongly to impose the condition" depends on which DOF you are looking at, and no single number is right for all of them. After normalisation the question is the same for every row, and the answer is the one JOREK already uses for every Dirichlet. **The scheme has no free parameter** — that is a direct consequence of this step, not a happy accident.

A degeneracy floor at $10^{-14} D_{\max}$ guards against a row with essentially no boundary support, where $1/D_a$ would manufacture an enormous row out of noise. In production it never triggers: the diagnostic reports `204 accumulated, 204 replaced, 0 below the floor` every step.

6.4 Ingredient 3: a trust region on the residual only

The residual $z_j^{\mathrm{sh}} - z_j$ is unbounded on the electron branch. At $u = 0$ the characteristic sits at $X = -\Lambda$ with $f = 1 - e^{\Lambda} \approx -19$, so a restart whose wall is not already near floating asks the constraint for $\sim 20\, j_{\mathrm{sat}}$ of driving force on the very first step. Measured doing exactly that: $e\Phi/kT_e$ collapsed to 0.00/0.01/0.03, 100 % of the area electron-limited, $I_{\mathrm{sheath}} = -12$ kA against $I_{\mathrm{Amp}} = +1$ kA, blow-up in four steps.

The residual is therefore passed through

$$r \;\longmapsto\; \frac{r}{1 + |r|/(r_{\max}\, z_j^{\mathrm{sat}})}, \tag{19}$$

with $r_{\max} =$ `sheath_weak_rmax` (default 2). This map is the *identity* for $|r| \ll r_{\max} z_j^{\mathrm{sat}}$, so the fixed point and the local convergence rate are untouched, and it saturates at $r_{\max} z_j^{\mathrm{sat}}$ however far off the state starts. An algebraic rather than a tanh clip is used deliberately: the derivative of (19) decays only algebraically, whereas a tanh's exponential decay would leave the row effectively explicit during the approach.

The Jacobian is deliberately *not* bounded — see the trap box in Section 3.6.

6.5 Ingredient 4: the validity weight

Where the plasma demands $|z_j/z_j^{\mathrm{sat}}|$ far outside what the characteristic can deliver, the relation carries no information and driving z_j toward it is actively destructive. Measured: a local j_{sat} collapse took the inner target from $\max|j/j_{\mathrm{sat}}| = 13 \to 18.7 \to 354$ in three steps and ended an otherwise healthy 550-step run.

The trust region cannot cover this, because it bounds the residual by $r_{\max} z_j^{\mathrm{sat}}$ locally, which vanishes together with z_j^{sat}. A separate smooth weight is applied,

$$w = \left[1 + \left(\frac{|z_j/z_j^{\mathrm{sat}}|}{\mathcal{R}_{\max}}\right)^4\right]^{-1}, \tag{20}$$

with $\mathcal{R}_{\max} =$ `sheath_zj_ratio_max`. It is monotone, equals 1 to within 6 % below $\frac{1}{2}\mathcal{R}_{\max}$, and — critically — is applied to F_a, D_a *and* the Jacobian columns alike, so the normalised row stays a consistent weighted residual and simply fades out at pathological points.

What "fading out" falls back to

A faded row leaves the assembled equation in place — which, per Section 5.2, is the *incomplete* one whenever `natural%zj` is off. The fallback is therefore not neutral. This was stated too optimistically in an earlier version of this document; setting `natural%zj` on the sheath types removes the hazard.

6.6 On under-relaxation

A parameter `sheath_weak_relax` $= \theta$ scales the state columns and the residual in (17) while leaving the z_j column (and hence the D_a normalisation) untouched, so the row reads $z_j^{\text{new}} = z_j^{\text{old}} + \theta(z_{j,\text{lin}}^{\text{sh}} - z_j^{\text{old}})$.

Leave it at 1. It was added to damp a period-2 alternation around step 568; that alternation turned out to be a transient of the relaxation phase and cleared itself by step 3900. Head-to-head at timeout, $\theta = 0.5$ was worse on every metric (Table 2) — under-relaxation makes the row *lag* a state that is still slowly evolving, so it never catches up.

6.7 Where the condition is attached, and why `natural%zj` is required

The sheath supplies one scalar relation per boundary point. It could go on the vorticity (charge continuity) equation as a surface flux, or on the current definition $z_j = \Delta^*\psi$ as an algebraic constraint. Both were built; the distinction is not cosmetic.

In `model600` the vorticity equation is assembled in *strong* form, so per Section 3.4 it has no boundary flux for a surface term to replace and adding one injects a spurious source. The route pursued here attaches the characteristic to the current definition instead, putting the condition on the variable the characteristic actually predicts.

A subtlety here falsified an earlier assumption. It was believed `dirichlet%zj` could stay on, on the grounds that the frozen z_j trace cancels between the volume term and the surface flux. The diagnostic refuted this directly: with the Dirichlet on, I_{Amp} stayed constant to four digits for a whole run while I_{sheath} ran from -15540 A to -23 A. The sheath adapted all the way to floating, the plasma current could not move at all, and u diverged trying to reconcile them.

> The j–V loop is *open* whenever z_j at the wall is pinned. The boundary condition can then only move the potential — and if the delivered current exceeds j_{sat}, no potential satisfies the characteristic.

What that measurement shows is that `dirichlet%zj` must be *off*. Whether the surface term must additionally be restored depends on the route, and the two differ:

- On the **natural%u** route nothing replaces the boundary z_j rows, so releasing the Dirichlet leaves them with an incomplete weak form. `natural%zj = .true.` is then genuinely required, coupling z_j at the boundary to $\nabla\psi\cdot\boldsymbol{n}$ — the *normal* derivative of ψ, which `dirichlet%psi` does not pin, since that fixes the value and the tangential derivative only.
- On the **weak** route the boundary z_j trace rows are *replaced* by the characteristic, so the surface term would only contribute to rows that are then overwritten. The loop closes through the constraint itself: z_j at the wall is not pinned, it is set to $z_j^{\text{sh}}(u, \rho, T_i, T_e)$, and the interior $z_j = \Delta^*\psi$ equation makes ψ's normal derivative follow. Measured: `natural%zj` left off ran 3900 steps at 0.00 % closure. It is **not** required here.

The one thing it still touches on the weak route is that the trace space spans only the value and tangential DOFs, so the z_j rows at the *normal*-derivative DOFs of boundary nodes are not replaced and keep an incomplete weak form without it. That is second order and was not measurable at 0.00 % closure, but it is the reason to enable it if a row ever falls below the accumulator's degeneracy floor.

6.8 Required namelist configuration

```
bcs(1)%sheath_zj_weak = .true. ! the weak Galerkin route (this work)
bcs(1)%dirichlet%zj = .false. ! REQUIRED - a pinned zj opens the j-V loop
bcs(1)%natural%zj = .false. ! optional here; see Section 5.7
```

```
bcs(1)%dirichlet%u = .false. ! u is free, set by charge continuity at the wall
bcs(1)%dirichlet%w = .true. ! KEEP; leave natural%w = .false.
bcs(1)%mach1 = .true. ! j_sat assumes Mach 1 here (default for type 1)
bc_natural_open = .true. ! boundary integrals live in that branch
```

Listing 1: Enabling the weak sheath condition on the strike-point boundary types.

At least one boundary type must keep `dirichlet%u = .true.` as a gauge — typically the main chamber wall. The reason: u enters the vorticity equation only through its gradient, and the sheath term loses its grip on u in ion saturation, so without an anchor somewhere the constant mode of u is undetermined.

7 Implementation

7.1 Module map and call ordering

File	Role
`model600/mod_sheath_bc.f90`	All sheath physics: normalisation, Λ, the X-limiter, and `sheath_current`, returning z_j^{sh}, z_j^{sat}, X and four derivatives. Stateless.
`model600/mod_boundary_matrix_open.f90`	Boundary element assembly. Evaluates the characteristic at each boundary Gauss point, forms D_a, F_a and the Jacobian blocks.
`model600/mod_sheath_trace.f90`	The accumulator: sums contributions across adjacent edges, normalises by D_a, writes the rows with `zbig`.
`model600/mod_boundary_conditions.f90`	Owns `a_mat` and `RHS_loc`; calls `sheath_trace_apply`. Also hosts the legacy nodal route.
`model600/mod_sheath_diag.f90`	Diagnostics: currents, $e\Phi/kT_e$ statistics, inner/outer split, nodal and weak residuals.
`matrix/construct_matrix_mod.f90`	Resets the accumulators before the element loop; reports afterwards.

The ordering within one matrix construction matters:

1. `sheath_diag_reset`, `sheath_trace_reset`;
2. the (OpenMP-threaded) element loop, calling `boundary_matrix_open` → `sheath_trace_add`;
3. `sheath_diag_report`;
4. `boundary_conditions` → `sheath_trace_apply` → `sheath_trace_report`.

7.2 Why an accumulator is needed at all

Rows cannot be written during the element loop. A trace DOF sits at a node shared by *two* adjacent boundary edges belonging to different elements, and normalising by D_a requires the *completed* sum over both. So contributions are accumulated first, keyed by global DOF index, and written only after the loop finishes.

```
integer, parameter :: st_max_row = 4000 ! trace DOFs per rank; 204 observed
integer, parameter :: st_max_col = 64 ! 3 nodes x 2 trace DOFs x 5 vars = 30

integer, save :: st_n, st_row(st_max_row), st_nc(st_max_row)
real*8, save :: st_D(st_max_row), st_F(st_max_row)
integer, save :: st_col(st_max_col, st_max_row), st_var(st_max_col, st_max_row)
real*8, save :: st_val(st_max_col, st_max_row)
```

Listing 2: The accumulator state (`mod_sheath_trace.f90`).

Overflow of either dimension is *fatal* rather than a silent truncation, on the grounds that a truncated row is a wrong equation rather than a missing one. The application step is then simply

```
sc = 1.d0 / st_D(is)
do ic = 1, st_nc(is)
  call boundary_conditions_add_one_entry( &
        st_row(is), var_zj, in, st_col(ic,is), st_var(ic,is), in, &
        zbig * st_val(ic,is) * sc, index_min, index_max, a_mat)
enddo
call boundary_conditions_add_RHS( &
        st_row(is), var_zj, in, index_min, index_max, RHS_loc, &
        zbig * st_F(is) * sc, a_mat%i_tor_min, a_mat%i_tor_max)
```

Listing 3: Normalisation and row writing (`sheath_trace_apply`).

7.3 Thread safety and the shared-node guard

The element loop is OpenMP-threaded and every `st_*` array is module-level shared state that `sheath_trace_add` both searches and extends. Without a critical section two threads race on the counter and write past each other — a crash, not a wrong answer. The routine body is wrapped in `!$omp critical`.

A second subtlety: `weak_sheath_zj` is an `.or.` over the edge's two nodes, so an edge with *one* sheath node is integrated in full. That is correct — those Gauss points are on the sheath surface. But the hand-off must *not* then write a replacement row for the other node if its own type has no sheath, because that node still carries `dirichlet%u`. Constraining $z_j = z_j^{\mathrm{sh}}(u)$ where u is frozen is not a boundary condition: u cannot respond, so the row simply injects whatever current the characteristic returns at the gauge potential ($u = 0$ gives $\Phi = 0$, $X = -\Lambda$, $f \approx -19$).

7.4 Known limitation: MPI

Rows are written only by the rank that owns them, so a trace DOF whose two adjacent edges live on different ranks is assembled from the local edge alone. Both F_a and D_a lose the same contribution, so the *normalised* row remains a valid weighted-residual statement — just tested against a function supported on one edge rather than two. A full fix needs a halo exchange.

Empirically the effect is not measurable at production sizes: 16 and 32 ranks gave identical output to four significant figures with the same 204 trace rows. That is a useful decomposition-independence check, not a proof.

8 Results

All results are for an ASDEX Upgrade X-point geometry (shot 38773), `model600`, $n_{\mathrm{tor}} = 1$, boundary type 1.

8.1 Convergence

At the final state I_{sheath} and I_{Amp} agree to four significant figures on both targets (inner $-281.7/-281.7$ A, outer $-402.2/-402.2$ A) and on the wall total ($-683.9/-683.9$ A). The converged *nodal* route left 48 % on the inner target and 4.1 % on the outer.

That $\varepsilon_{\mathrm{gauss}}$ collapses too (1.99 to 9.0×10^{-3}) was better than expected: a Galerkin projection only has to remove the component of the mismatch *lying in* the trace space, and is under no obligation to reduce the pointwise error. That it does anyway indicates the trace space represents z_j^{sh} well on this mesh.

Step	$\varepsilon_{\text{weak}}$	$\varepsilon_{\text{gauss}}$	INNER closure	OUTER closure	I_{wall}
0	1.759	1.990	—	—	—
9	0.049	0.071	4.1 %	0.05 %	—
30	9.9×10^{-4}	9.7×10^{-3}	0.014 %	0.012 %	−136 A
568	2.0×10^{-2}	3.9×10^{-2}	0.6–1.6 %	0.2–0.3 %	−2950 A
∼3900	$\mathbf{6.5 \times 10^{-4}}$	$\mathbf{9.0 \times 10^{-3}}$	**0.00** %	**0.00** %	−684 A

Table 1: Convergence. "Closure" is $|I_{\text{sheath}} - I_{\text{Amp}}|/|I_{\text{sheath}}|$. The excursion at step 568 is the plasma relaxing on a transport timescale, not a degradation of the constraint; it recovers without intervention. The run ended on wall-clock timeout, not instability.

Metric at timeout	$\theta = 1.0$	$\theta = 0.5$
$\varepsilon_{\text{weak}}$	$\mathbf{6.5 \times 10^{-4}}$	6.3×10^{-3}
$\varepsilon_{\text{gauss}}$	$\mathbf{9.0 \times 10^{-3}}$	1.5×10^{-2}
Target closure	**0.00** %	0.05–0.12 %
$e\Phi/kT_e$ max	**6.56** (falling)	7.32 (rising)
`construct_matrix` time	**0.276** s	0.334 s

Table 2: Under-relaxation is counterproductive. Both ran to timeout without crashing.

8.2 Under-relaxation comparison

8.3 Physical state

At the converged state the wall carries −684 A net, inner −281.7 A and outer −402.2 A. Mean $e\Phi/kT_e = 2.46$ against $\Lambda = 3.19$, with an inner/outer contrast of 2.52 versus 2.38. Some 8.3 % of the wetted area sits on the electron-side limiter, with $\max|j/j_{\text{sat}}| = 13.1$, steady.

Two cautions. The large in–out asymmetries seen during the transient (a factor 3.5 at step 30, a reversed contrast at step 568) are *not* representative and should not be quoted. And a small contrast in $e\Phi/kT_e$ does not imply a small potential difference: $\Phi = (e\Phi/kT_e) \cdot kT_e/e$, so at steady state the asymmetry lives in the target T_e contrast rather than in the normalised drop.

8.4 What is *not* verified

Worth stating plainly. The diagnostics test that the solver satisfies $z_j = z_j^{\text{sh}}(u)$; they do not test that `sheath_current` is right, because I_{sheath} and I_{Amp} share a normalisation (Section 2.5). Four tests would close that gap:

- **Circular limiter, uniform T_e.** No thermoelectric drive means no current, so the characteristic requires $X = 0$, i.e. $e\Phi/kT_e = \Lambda$ *exactly*. Reads straight off the existing log, no new diagnostic.
- **Λ-scan.** $\Phi_f = \Lambda kT_e/e$ is exactly linear, so Φ in volts must scale with slope T_e/e. This is the absolute check a units error cannot survive.
- **j–V collapse.** Plot j/j_{sat} against X for every boundary point, computed externally in SI. All points must land on $f(X)$ with no free parameters.
- **h-convergence.** Refine the boundary and watch the closure error. A plateau means the trace assembly is inconsistent.

9 Explaining this in five minutes

If you have to describe this to a colleague at a whiteboard, this is the shape of it.

1. **The physics.** A wall in contact with plasma floats to $\Phi = \Lambda T_e/e$. Since T_e varies across a divertor target, so does Φ, which means there is an electric field at the target and therefore an $\boldsymbol{E} \times \boldsymbol{B}$ drift. The old boundary condition froze Φ at zero and forbade all of that.

2. **What we impose.** The Langmuir characteristic, in the forward direction: given the potential, what current flows. Written the other way it is singular exactly where divertor plasmas sit.

3. **The finite-element twist.** JOREK's solution is not values on a grid — it is coefficients multiplying cubic basis functions, four per node. On a boundary edge the solution is a cubic determined by four of them: value and tangential slope at each end.

4. **Why the obvious thing fails.** Imposing the characteristic at nodes controls one coefficient out of four. The cubic between the nodes goes where it likes — and since the wall current is an *integral* over that cubic, so does the current. Measured: satisfied at the nodes to 2×10^{-4}, wrong in the integrated current by 48 %.

5. **The fix.** Impose it as an integral instead: require the mismatch to be orthogonal to the trace space. Then the closure holds by construction rather than by luck.

6. **The three things that made it work.** *Replace* the boundary rows rather than adding a penalty to them, because the equation being added to is incomplete. *Normalise* each row by its own diagonal, because derivative rows are 10^{10} times smaller than value rows and no single coefficient fits both. *Bound the residual* but not the Jacobian, because Newton solves $\mathsf{J}\,\mathrm{d}x = F$ and shrinking J makes the step larger.

7. **The result.** Wall current closes to four digits on both targets, no tuning parameter, 3900 steps stable.

10 Limitations and outlook

Axisymmetric only. The accumulator holds one toroidal index, checked at setup. Extending to $n_{\mathrm{tor}} > 1$ needs a toroidal dimension and, more substantially, thought about whether the trace projection should couple harmonics.

Quasi-Newton. Omitting the ψ column (Section 3.6) freezes the boundary geometry within an iteration. This is the most plausible remaining source of stiffness and the first thing to add if convergence degrades.

Two sheath surfaces. Enabling the condition on two boundary types simultaneously has not been made to work: u then closes a circuit between them whose only impedance is the sheath itself, which is nonlinear with near-zero differential conductance on both branches, and nothing damps the loop. Four attempts failed, including one genuine bug fix that changed nothing.

Missing physics elsewhere. Two gaps found in passing, independent of the boundary condition but relevant to divertor asymmetry: `model600` lacks the ∇B terms in the energy equations, and lacks the thermoelectric heat-flux divergence $\nabla\cdot(0.71\,T_e j_\parallel/e)$. Both appear in SOLPS and both act in the in–out channel.

A note on method

The substantive lesson is Section 4: a constraint can be satisfied to four decimal places in the norm you are measuring and still be wrong by tens of percent in the quantity you care about.

The diagnostic that resolved it — instrumenting the weak residual as a pure observable, *before* changing any code — cost far less than the sequence of tuning experiments it made unnecessary.

Glossary

Basis function N_a A fixed, known function attached to a degree of freedom. The solution is a weighted sum of these. Local: nonzero only near its own node.

Degree of freedom (DOF) One unknown number the code solves for. In JOREK, four per node per variable: value, two first derivatives, one mixed second derivative.

Trace The solution restricted to a boundary edge. For a bicubic it is a cubic fixed by four DOFs: value and tangential derivative at each endpoint.

Strong form The PDE as an equality at every point.

Weak form The PDE multiplied by a test function and integrated. Gives one equation per DOF and produces surface terms on integration by parts.

Natural boundary condition One imposed through the surface term of the weak form (Neumann-like). Needs no row surgery.

Essential boundary condition One imposed by overwriting rows (Dirichlet-like).

Galerkin projection Requiring the residual to be orthogonal to the space of test functions — i.e. zero as far as the discretisation can tell, rather than zero pointwise.

Mass matrix The matrix with entries $\int N_a N_b$. Here, its boundary restriction, whose diagonal D_a is used to normalise the rows.

Jacobian The matrix of $\partial(\text{residual})/\partial(\text{unknown})$ used to linearise a nonlinear equation for Newton's method.

Trust region A bound on how far one Newton step may move, applied here to the residual (the right-hand side), never to the Jacobian.

Conditioning How widely a matrix's scales are spread. Poorly conditioned matrices lose precision and defeat uniform scaling.

Quadrature / Gauss points The sample points at which integrals are evaluated numerically. They sit *between* the nodes, which is the entire subject of Section 4.

References

[1] P. C. Stangeby, *The Plasma Boundary of Magnetic Fusion Devices*, IOP Publishing, 2000. (Sheath characteristic, eq. 2.68.)

[2] M. Hölzl *et al.*, "The JOREK non-linear extended MHD code and applications to large-scale instabilities and their control in magnetically confined fusion plasmas", *Nucl. Fusion* **61** 065001 (2021).

[3] F. J. Artola *et al.*, on sheath boundary conditions for the electrostatic potential in JOREK. (Saturation current.)

[4] R. Chodura, "Plasma–wall transition in an oblique magnetic field", *Phys. Fluids* **25** 1628 (1982).